EDGE-AI · PREDICTIVE MAINTENANCE

# Intelligence at the machine, not in the cloud.

Detect a fault before it breaks · explain it in plain language · act autonomously — **with the cable unplugged.**

**On-device ML** | **Edge LLM diagnosis** | **Autonomous actuation**

Running live on a Raspberry Pi 5 “edge ECU” · LTTS Engineering Intelligence Hackathon 2026

THE PROBLEM · TODAY

# The data is born at the machine — but the intelligence is trapped in the cloud.

***That split is exactly backwards*** *— and it's the whole problem.*

Connected fleets and industrial machines generate gigabytes of sensor telemetry — but the intelligence to act on it lives in the cloud: costly to reach, and unavailable exactly where these assets operate.

- **Remote by nature** Highways, mines, farms, job sites — intermittent or expensive connectivity.
- **A losing choice** Stream a raw firehose you mostly discard, or run blind until something fails.
- **Preventable → unplanned** A slow wear-out becomes a stranded machine and an emergency callout.

THE CHALLENGE

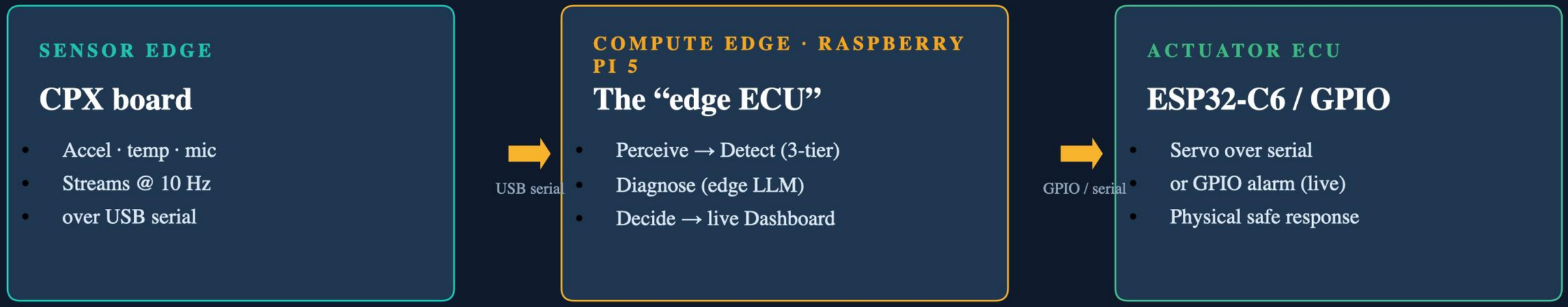
## Push the intelligence onto the machine itself.

Turn a live sensor stream into **action** — entirely on-device:

- Detect an impending fault
- Explain it in plain language
- Trigger a safe response autonomously
- Uplink only insight (~3 KB), not the raw firehose (~121 KB)

SOLUTION ARCHITECTURE

# A two-tier edge that mirrors a software-defined-vehicle stack




**Ships only insight** ~3 KB — not raw telemetry ~121 KB per 30-min window **≈ 97% less uplink**

**Key architectural choice:** detection is the source of truth and runs with zero network/LLM dependency — the LLM is the *only* layer permitted to fail.

HOW IT WORKS

# One offline loop: perceive → decide → act

| | | |
|---|---|---|
| 1 | **Perceive** | CPX firmware streams accel/temp/mic over serial; the Pi parses it into frames. (Software path: reproducible J1939 CSVs.) |
| 2 | **Detect** | Three parallel detectors — fixed threshold, IsolationForest, CUSUM drift. ~4 ms/row, no GPU, no network. |
| 3 | **Diagnose** | A small local LLM (qwen2.5:1.5b via Ollama) writes a 2-sentence technician summary — deterministic template fallback. |
| 4 | **Decide → Act** | A pure decide() policy drives a physical actuator: GPIO alarm (live) or ESP32-C6 servo. Degrades to a logging mock. |

AI APPROPRIATENESS

# The right tool for each job — and we can defend every choice

CLASSICAL ML

## Detection — no LLM, on purpose

The safety path must be deterministic, fast, and incapable of hallucinating a fault or missing a red-line.

- Fixed threshold — cannot be talked out of a breach
- IsolationForest — multivariate outliers
- CUSUM — slow predictive drift
- No GPU · no network · no prompt

EDGE LLM

## Diagnosis — where language is the task

Turning a 4-channel numeric state into a sentence a technician can act on is a translation problem.

- Exactly what a small LM is good at
- Quarantined as the one fail-allowed layer
- Never gates detection or actuation
- Latency & non-determinism can't endanger safety

**AI where it adds value · determinism where uptime depends on it.**

AI IMPLEMENTATION · MODEL CHOICE

# Why qwen2.5:1.5b — chosen by measurement, not reputation

## THE ON-DEVICE SWEEP

Each candidate timed on the Pi 5 on the real anomaly→summary prompt (benchmark_edge_llm.py). Deciding metric: time to a 2-sentence summary vs. a 10 s cutoff.

| | | |
|---|---|---|
| `qwen2.5:0.5b` | *fast, but lower quality* | ✓ |
| **`qwen2.5:1.5b`** | *speed / quality sweet spot* | ✓ **CHOSEN** |
| `llama3.2:1b` | *under the cutoff* | ✓ |
| `gemma2:2b` | *under the cutoff, larger* | ✓ |
| `llama3.2:3b` | *too slow on the Pi* | ✗ |

## WHY IT'S THE RIGHT PICK

### Measured, not assumed

The benchmark times each model on the exact prompt the demo uses — the choice is data, not a spec-sheet.

### Runs ON the Pi (Option A)

~6–12 s live, under the 10 s cutoff → no gateway, no cloud, no network in the loop.

### Speed / quality sweet spot

Big enough for a coherent 2-sentence diagnosis, yet 4-bit-quantized & CPU-only — shares 4 cores with the stream loop + Flask. 3B was too slow; 0.5B weaker.

### Small is safe here

A deterministic template fallback covers the rare slow call, so we optimize for usually-fast + coherent, not worst-case.

***Why not a bigger model?*** *It can't meet the on-device budget — and the task doesn't need it. We measured both.*

AI IMPACT · MEASURED RESULTS

# Every number reproducible on this Pi 5

**193 s**

**PREDICTIVE LEAD**

CUSUM flagged oil-pump drift before the hard safety threshold broke

**~97%**

**LESS UPLINK**

~121 KB raw telemetry → ~3 KB AI-summarized insight per 30-min window

**13 / 13**

**FAULT WINDOWS**

0 false alarms · 0 soft flags · 90 noise blips debounced on the healthy machine

**~4 ms**

**PER ROW**

~250× real-time headroom at 1 Hz — streaming inference, not batch

**6–12 s**

**PER DIAGNOSIS**

Median edge-LLM latency for a 2-sentence maintenance summary

**$0**

**CLOUD BILL**

No contract, no recurring inference cost, no connectivity requirement

*Zero alert fatigue — every alert a human sees is real.*

ENGINEERING & BUILD QUALITY

# Real hardware, verified — not a slideware demo

### Runs on real silicon

CPX firmware on CircuitPython 10.2.1; a 447-frame live capture confirms all four signals. We even found & fixed a platform bug (cp.sound_level unsupported → drive the PDM mic directly).

### Loop closes at the sensor

On a confirmed fault the Pi writes back over serial → CPX flashes 10 NeoPixels + sounds its speaker. Fixed a second platform trap: CircuitPython input() froze the sample loop; replaced with a raw byte-buffer read.

### Mobile-first deployment

The Pi hosts its own WiFi hotspot + local DNS — a phone gets the live dashboard at a plain hostname, no IP, no port, zero external network.

### Graceful degradation everywhere

GPIO → logging mock; LLM → template on timeout; serial skips malformed lines; unconfirmed events get an honest ‘monitor’ note. No silent failure mode.

### Validated, not hand-tuned

All hysteresis params fixed by a sweep across four scenarios (healthy = 0 events, every fault caught, ≥60 s lead). Fixed seeds regenerate every dataset identically.

*“We found and fixed two real CircuitPython platform bugs getting this working — this is a built system, not a mockup.”*

DIFFERENTIATION

# The category is mature — we win on a specific gap

| Deployed tech | What it does | Where we differ |
|---|---|---|
| **Fleet telematics** Samsara · Geotab · Motive | Stream telemetry to cloud dashboards | **Cloud-centric, needs connectivity** |
| **Industrial PdM** Augury · Uptake · Senseye | Edge vibration + cloud diagnosis, stationary | **Diagnosis on-device, for mobile/J1939 assets** |
| **OEM telematics** VisionLink · Komtrax · JDLink | Fault codes (DTCs) + cloud dashboards | **Plain-language diagnosis + autonomous action** |
| **On-board diagnostics** OBD / DTC | Fire a code at/after a threshold breach | **CUSUM is predictive — flags drift before** |

**1. Diagnosis on the edge**

Generative language on-device, offline

**2. The entire loop is offline**

Detect → explain → act, zero network

**3. Predictive + autonomous**

Drift caught before the hard bound

BUSINESS RELEVANCE & VIABILITY

# Offline is the buying reason — and the economics are hard numbers

## WHO BUYS IT

- **Commercial & off-highway OEMs** J1939-native — the LTTS-shaped client
- **Remote-asset operators** Mining, construction, ag, marine — offline is the reason
- **Long-haul fleets** ~97% bandwidth cut × thousands of vehicles
- **Industrial rotating machinery** Pumps, motors, compressors — same architecture

## PILLAR FIT

Mobility | Sustainability | Tech | Physical AI | Agentic AI

## VIABILITY & COST

**~$80 Raspberry Pi 5 + ~$30 sensor board**

Prebuilt arm64 apt packages · free local model. **No cloud contract · no recurring inference bill · no connectivity requirement.**

## ROI

Unplanned downtime runs to tens of thousands of dollars per hour. **PdM is widely cited to cut downtime ~30–50%** — we attack both downtime and diagnostic-labor cost.

ROADMAP

# Differentiators no fault-code system matches

IN PROGRESS

## Remaining Useful Life (RUL)

Reuse the CUSUM drift slope to estimate *"oil pressure crosses the safety bound in ~4 h at the current wear rate."*

Turns lead-time into a plannable number.

IN PROGRESS

## On-device service-manual RAG

Ground the LLM in the machine's manual so the diagnosis names the likely **part, procedure, and torque spec** — all offline.

Everything on the earlier slides is **built & verified** — live J1939 pipeline, real CPX hardware capture, streaming dashboard, physical actuation, mobile hotspot view. These two are the **roadmap differentiators**.

THE ONE-LINE PITCH

# We don’t just detect it minutes early — we tell you what it is, which part, how long you’ve got, and we do it with the cable unplugged.

**Predictive**
drift caught 193 s early

**Diagnostic**
plain-language, on-device

**Autonomous**
closed to physical actuation

**Offline**
zero network dependency

Edge-AI Predictive Maintenance · running live on a Raspberry Pi 5 · Thank you

# Speaker Notes & Q&A

Presenter reference — talking points and judge-Q&A answers for each slide.

### SLIDE 1 Title — Intelligence at the machine, not in the cloud

Our project puts the whole intelligence loop — detect, diagnose, decide, act — onto the machine itself. No cloud, no network dependency. Everything you'll see is measured on this Raspberry Pi 5 and reproducible.

### SLIDE 2 The Problem — data born at the machine, intelligence trapped in the cloud

The core problem: telemetry is everywhere, but the brains are in the cloud — unreachable where these machines actually run. The challenge is to move the intelligence onto the device and never depend on a network to act.

### SLIDE 3 Solution Architecture — the two-tier edge

Three physical devices: a sensor board, the Pi as compute edge, and an actuator. It mirrors a real SDV stack: ECU to domain controller to actuator. Crucially we ship only the AI summary upstream — a 97% bandwidth cut we actually measured.

### SLIDE 4 How It Works — one offline loop: perceive → decide → act

The whole loop runs on the device: perceive the sensor stream, detect anomalies with classical ML, diagnose in plain language with a small LLM, then decide and physically act. Detection never waits on the LLM or the network.

### SLIDE 5 AI Appropriateness — right tool for each job

We deliberately don't use an LLM for detection — that path must be deterministic and fast. The LLM only does what language models are actually good at: translating numbers into a technician-ready sentence. And it's quarantined so it can never endanger the safety loop.

Q&A — 'What if the LLM gives a wrong answer?' Two cases. (1) It fails loudly (error/timeout): we drop to a deterministic templated summary built from the detected facts — blunter but correct. (2) It hallucinates a plausible-but-wrong diagnosis: that's an advisory-text error, not a safety event. decide() acts on the CONFIRMED sensor event, not on the model's words, so a wrong sentence can't miss a fault, can't trigger or suppress the actuator, and can't corrupt the record — the real signal, values and lead time are shown right next to the prose to contradict it. We also constrain the prompt to name the flagged signal (an overspeed event won't get an overheat diagnosis), and unconfirmed events get an honest 'monitor' note. Worst case = one misleading sentence beside the real numbers. If pushed: yes it could briefly mislead a tech — which is why on-device service-manual RAG is on the roadmap, to anchor the language to real parts/procedures.

### SLIDE 6 Model Choice — why qwen2.5:1.5b

Backup slide for the model-choice question. The key point: we didn't pick the biggest model that fit — we benchmarked candidates on-device on the real prompt and picked the one that meets the latency budget with enough quality. qwen2.5:1.5b runs on the Pi in ~6–12 s (llama3.2:3b misses the 10 s cutoff), is quantized and CPU-only so it shares cores with the rest of the pipeline, and the deterministic fallback lets us stay small because it absorbs the rare slow call.

### SLIDE 7 AI Impact — measured results

These are the headline numbers, all measured and reproducible. 193 seconds of predictive lead on real slow wear translates to hours or days of planned versus emergency maintenance. 97% less uplink. And zero false alarms across every scenario.

### SLIDE 8 Engineering & Build Quality — real hardware, verified

This is a working build, not slideware. Real firmware, a 447-frame live capture, two genuine platform bugs found and fixed, mobile deployment over the Pi's own hotspot, and graceful degradation at every layer so it never fails silently.

### SLIDE 9 Differentiation — we win on a specific gap

We don't claim nothing like this exists — the category is mature. We win on a specific gap: diagnosis generated on the device, the entire loop offline, and a predictive tier that flags drift before a fault code would ever trip.

### SLIDE 10 Business Relevance & Viability

The customers who need this are the ones where offline is the buying reason. Economics are concrete: ~$110 of hardware, no cloud bill, and it attacks downtime that costs tens of thousands per hour. It hits every judging pillar.

### SLIDE 11 Roadmap — RUL + on-device manual RAG

Two roadmap differentiators no fault-code system can match: RUL projection that turns our drift slope into a plannable hours-to-failure number, and on-device manual RAG so the diagnosis names the exact part and torque spec. Everything shown before this slide is already built and verified.

### SLIDE 12 The One-Line Pitch

Our one-line pitch: we don't just detect it early — we tell you what it is, which part, how long you've got, and we do it with the cable unplugged. Predictive, diagnostic, autonomous, and fully offline. Thank you.